

# Informe de Validaciones

Proteccion contra duplicados y reglas de negocio

---

Fecha: 12/06/2026 11:44

Proyecto: Simulador Mundial 2026

CDA Soluciones Confiables

## 1. Resumen Ejecutivo

---

Este informe documenta todas las validaciones implementadas en el Simulador Mundial 2026 para garantizar la integridad de los datos y prevenir duplicados en las entidades Team y Player. Se implementaron 3 capas de proteccion: constraints a nivel base de datos, validaciones en la capa de servicios, y un handler global de errores de integridad.

Total de tests: 82 | Estado: 82/82 PASS | Fecha: 12/06/2026

### Capas de proteccion

- Capa 1: UniqueConstraint en base de datos (SQLite/SQLAlchemy)
- Capa 2: Validaciones en services (TeamService, PlayerService)
- Capa 3: Exception handler global para IntegrityError en main.py

## 2. Validaciones - Team

---

### 2.1 Reglas implementadas

#### Codigo unico

No pueden existir dos equipos con el mismo codigo (409 Conflict)

#### Nombre unico

No pueden existir dos equipos con el mismo nombre (409 Conflict)

#### Formato de codigo

El codigo debe ser exactamente 3 letras mayusculas (400 Bad Request)

#### Nombre no vacio

El nombre del equipo no puede estar vacio (400 Bad Request)

#### Actualizar codigo duplicado

Al cambiar codigo, validar que no exista otro equipo con ese codigo (409)

#### Actualizar nombre duplicado

Al cambiar nombre, validar que no exista otro equipo con ese nombre (409)

### 2.2 Tests especificos

- test\_create\_team - Creacion basica de equipo (201)
- test\_create\_team\_duplicate\_code - Codigo duplicado (409)
- test\_create\_team\_duplicate\_name - Nombre duplicado (409)
- test\_create\_team\_code\_too\_short - Codigo muy corto (400)
- test\_create\_team\_code\_too\_long - Codigo muy largo (400)
- test\_create\_team\_code\_with\_numbers - Codigo con numeros (400)
- test\_create\_team\_empty\_name - Nombre vacio (400)
- test\_update\_team - Actualizacion basica (200)
- test\_update\_team\_duplicate\_name - Actualizar a nombre existente (409)
- test\_update\_team\_duplicate\_code\_different\_team - Actualizar a codigo existente (409)

## 3. Validaciones - Player

---

### 3.1 Reglas implementadas

#### Nombre unico por equipo

No pueden existir dos jugadores con el mismo nombre en el mismo equipo (409)

#### Nombre valido entre equipos

El mismo nombre de jugador puede existir en diferentes equipos (201)

#### Posicion valida

La posicion debe ser una de: GK, DF, MF, FW (400)

#### Nombre no vacio

El nombre del jugador no puede estar vacio (400)

#### Maximo 23 jugadores

Un equipo no puede tener mas de 23 jugadores (400)

#### Team existente

El team\_id debe referenciar a un equipo existente (404)

#### Actualizar nombre duplicado en team

Al cambiar nombre, validar duplicado en el mismo equipo (409)

#### Mover jugador a team con duplicado

Al cambiar team\_id, validar duplicado en el destino (409)

### 3.2 Tests especificos

- test\_create\_player - Creacion basica (201)
- test\_create\_player\_invalid\_position - Posicion invalida (400)
- test\_create\_player\_team\_not\_found - Team inexistente (404)
- test\_create\_player\_duplicate\_in\_same\_team - Duplicado en mismo equipo (409)
- test\_create\_player\_same\_name\_different\_team - Mismo nombre en otro equipo (201)
- test\_create\_player\_empty\_name - Nombre vacio (400)
- test\_max\_players\_per\_team - Supera maximo 23 jugadores (400)
- test\_update\_player\_duplicate\_name\_in\_team - Actualizar a nombre duplicado (409)
- test\_update\_player\_move\_to\_team\_with\_duplicate - Mover a team con duplicado (409)

## 4. Cobertura por Capas

---

### 4.1 Base de Datos

Archivo: models/player.py, models/team.py

- Team.name: UNIQUE constraint (SQLAlchemy)
- Team.code: UNIQUE constraint (SQLAlchemy)
- Player: UniqueConstraint('name', 'team\_id', name='uq\_player\_per\_team')

4.2 S

Archivos: services/team\_service.py, services/player\_service.py

- TeamService.create(): valida code unico, name unico, formato code 3 letras, name no vacio
- TeamService.update(): valida code/name unico al modificar (excluyendose a si mismo)
- PlayerService.create(): valida team existente, posicion valida, name+team\_id unico, max 23 jugadores, name no vacio
- PlayerService.update(): valida team existente, posicion valida, name+team\_id unico al modificar

4.3 H

Archivo: main.py

- IntegrityError handler: captura errores de constraints no manejados y devuelve 409 en vez de 500
- Actua como fallback ante race conditions

4.4 T

Archivo: tests/test\_frontend\_smoke.py

- test\_frontend\_main\_simulation\_flow - Flujo completo de simulacion
- test\_api\_duplicate\_team\_code\_via\_browser -Codigo duplicado via API browser
- test\_api\_duplicate\_team\_name\_via\_browser - Nombre duplicado via API browser
- test\_api\_duplicate\_player\_in\_team\_via\_browser - Player duplicado en team via browser
- test\_api\_player\_same\_name\_different\_team\_via\_browser - Mismo player en otro team (valido)

## 5. Resultados de Tests

### 5.1 API Tests (pytest)

Comando: `python -m pytest tests/ --ignore=tests/test_frontend_smoke.py -v`

Resultado: 82 tests PASS, 0 FAIL

### 5.2 Playwright Tests

Comando: `python -m pytest tests/test_frontend_smoke.py -v --headed`

Resultado: 5 tests (requieren servidor FastAPI corriendo)

### 5.3 Cobertura de reglas

| Regla de Negocio                     | Capa                  | Estado |
|--------------------------------------|-----------------------|--------|
| Team code unico en BD                | BD (UNIQUE)           | PASS   |
| Team name unico en BD                | BD (UNIQUE)           | PASS   |
| Player name+team_id unico en BD      | BD (UniqueConstraint) | PASS   |
| Team code unico en API               | Service               | PASS   |
| Team name unico en API               | Service               | PASS   |
| Formato code 3 letras                | Service               | PASS   |
| Team name no vacio                   | Service               | PASS   |
| Player name+team_id unico en API     | Service               | PASS   |
| Player posicion valida (GK/DF/MF/FW) | Service               | PASS   |
| Player name no vacio                 | Service               | PASS   |
| Max 23 jugadores por equipo          | Service               | PASS   |
| Team existente al crear player       | Service               | PASS   |
| IntegrityError handler global        | main.py               | PASS   |

## 6. Documentacion Generada

---

### **QA\_NUEVAS\_REGLAS.md**

Reglas de negocio detalladas, contratos API, capas de proteccion, edge cases

### **QA\_REPORT\_PLAYWRIGHT.md**

Reporte formal de QA con casos F1-F5, tabla de cobertura 9/9, conclusion GO

### **SESION\_COMPLETA.md**

Resumen de toda la sesion de trabajo, cambios realizados y comandos utiles

### **allure-report/index.html**

Reporte HTML interactivo de Allure con resultados de 82 tests

### **VALIDACIONES.pdf**

Este documento - Informe completo en PDF

## 7. Conclusion

---

Se implementaron exitosamente 13 reglas de validacion distribuidas en 3 capas de proteccion (Base de Datos, Servicios, Handler Global).

Todos los tests (82/82) pasan correctamente, cubriendo escenarios positivos, negativos y de borde. Se incluyen 5 tests E2E via Playwright que validan el comportamiento desde el navegador.

La arquitectura de proteccion contra duplicados garantiza que:

- No existan equipos con codigo o nombre duplicado
- No existan jugadores con el mismo nombre dentro del mismo equipo
- Los codigos de equipo tengan formato estandarizado (3 letras)
- Los equipos no superen los 23 jugadores
- Cualquier error de integridad no manejado sea capturado y devuelva 409 en vez de 500

### **CONCLUSION: APROBADO (GO)**

Documento generado automaticamente el 12/06/2026 a las 11:44

CDA Soluciones Confiables - Simulador Mundial 2026